

Instituto Tecnológico y de Estudios Superiores de Monterrey

TC6001.570

Foundations of Artificial Intelligence

Project Assignment (Topic 2): Solving Job-Shop Scheduling via Search, Heuristics and Local Search

Team members:

A01986448

A01985570

A01633395

A01174432

Due date: October 30, 2026

Solving Job-Shop Scheduling via Search, Heuristics and Local Search

1. Introduction and problem definition

The Job-Shop Scheduling Problem (JSPP) is a classic combinatorial optimization problem classified as NP-hard, considered one of the most complex of its kind. In this system, scheduling tasks means assigning shared resources to different activities that compete for them. A set of jobs must be processed through dedicated machines, where each job requires a strict sequence of operations with predetermined processing times (Yamada & Nakano, 1997). The objective of this project is to implement and experimentally compare search (informed and uninformed) and local search approaches for classical JSP, with the aim of minimizing *makespan* under a rigorous and reproducible evaluation protocol.

2. Mathematic Formalization

Be a set of *n* jobs $\{J_j\}_{1\leq j\leq n}$ that must be processed on a set of *m* machines $\{M_r\}_{1\leq r\leq m}$, each job visits each machine exactly once. Each machine can process a maximum of one job at a time. Each job J_j has a fixed order of machines by which it must be processed. The processing of J_j in the machine M_r is called as operation O_{jr}. The operation O_{jr} requires the use of M_r for an uninterrupted duration p_{jr}, known as processing time. All durations are known in advance. A *schedule* is a set of completion times for each operation $\{C_{jr}\}_{1\leq j\leq n,1\leq r\leq m}$ (Yamada & Nakano, 1997).

If every schedule begins in a global time of $t = 0$, the main decision variable is the start time of each operation S_{jr}, such that $C_{jr} = S_{jr} + p_{jr}$. Any valid schedule must strictly satisfy three operational constraints:

- *Precedence:* The operations of the same job must be executed in their prescribed sequential order. An operation cannot start until the previous one of the same job has finished.
- *Machine capacity:* Two different operations on the same machine cannot overlap; conflicting jobs must wait until the resource is freed (Yamada & Nakano, 1997).
- *Non-preemption:* Once started, an operation O_{jr} requires the exclusive use of machine M_r until completion, without interruptions.

The total time required to complete all jobs is called *makespan*, denoted as L (Yamada & Nakano, 1997). The objective is to determine the schedule that minimizes L:

$$L = \max(C_{jr}) \quad \forall j, r$$

Additionally, a lower bound can be calculated for L. No schedule can finish before the longest job ends or the machine with the highest workload finishes. This bound is used to evaluate the quality of solutions in experiments.

3. Base case study ($3\ jobs \times 3\ machines$)

To illustrate the mechanics of the problem and validate the algorithmic implementations, a 3×3 base problem extracted from the literature was reproduced (Yamada & Nakano, 1997). This model defines the route and processing time (in parentheses) for $n = 3$ jobs on $m = 3$ machines:

- Job 1 (J_1): $M_1(3) \rightarrow M_2(3) \rightarrow M_3(3)$
- Job 2 (J_2): $M_1(2) \rightarrow M_3(3) \rightarrow M_2(4)$

- Job 3 (J_3): $M_2(3) \rightarrow M_1(2) \rightarrow M_3(1)$

A convenient way to visually represent the solutions for this and other JSSP problems in the experiments is using Gantt diagrams. Figure 1 illustrates the Gantt diagram generated by our implementation, displaying how operations for jobs J_1, J_2, and J_3 are allocated across machines over time to evaluate the resulting total completion time.

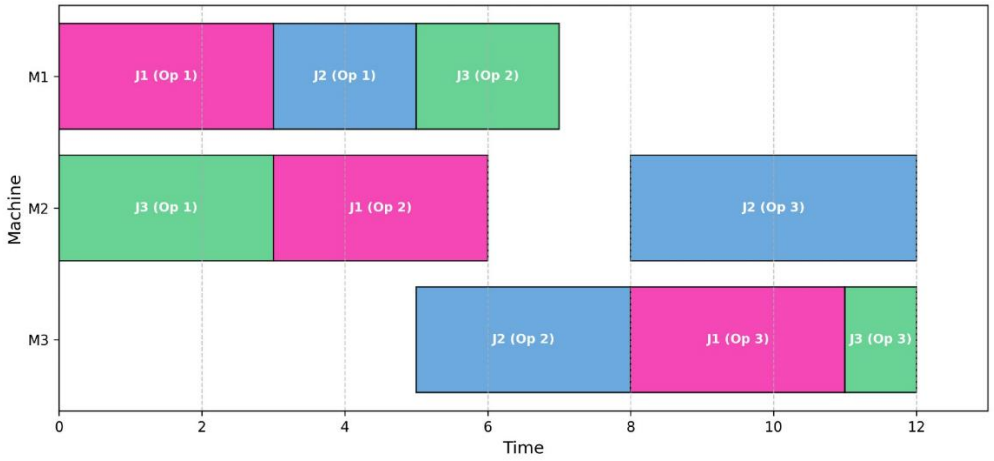


Figure 1. Gantt diagram representation for the 3x3 JSSP base instance.

3.1 Uninformed Search (baseline): Depth-First Search (DFS) Algorithm with Pruning

To establish a baseline algorithm a Depth-First Search (DFS) approach was implemented. In combinatorial optimization problems like Job-Shop Scheduling, DFS is widely used because it requires very little memory and it can quickly find valid schedules in the early stages of execution. Reaching a leaf node quickly allows the algorithm to find a complete sequence of operations. Although this initial schedule is rarely optimal, it immediately provides a maximum time reference known as an *Upper Bound* (UB). It represents the *makespan* of the current best solution and is crucial for reducing the search space in subsequent steps (Pinedo, 2008).

A pure DFS would exhaustively explore all possible combinations, wasting time on inefficient paths. Therefore, it is necessary to integrate a pruning mechanism. According to Mencía et al. (2015), in DFS algorithms applied to the JSSP, an intermediate state s can be pruned if its partial evaluation time $f(s)$ is greater than or equal to the current UB ($f(s) \geq UB$). In our implementation, this pruning technique was integrated directly into the successor generation loop:

- Global Upper Bound (UB): We use a variable named *best_makespan* that stores the total time of the best complete solution found so far.
- Partial Evaluation ($f(s)$): Before adding a new operation, we need to calculate its projected completion time (*end_time*). This value acts as a strict local *Lower Bound* for that specific branch.
- Dominance Rule: If the operation's completion time is greater than or equal to our current best time (*end_time >= best_makespan*), it is mathematically impossible for that path to yield a better global makespan (Mencía et al., 2015). Consequently, the algorithm discards this candidate state and skips adding it to the stack, significantly reducing memory usage and node expansions.

The following pseudocode formalizes the implemented logic, based on the bounded search methods proposed in the scheduling literature (Mencía et al., 2015; Pinedo, 2008):

```
Algorithm 1: DFS with Upper Bound Pruning for JSSP
Input: Initial state S_0
Output: Best schedule found and its makespan

Initialize OPEN_STACK = [S_0]
best_makespan (UB) = ∞
best_schedule = NULL

While OPEN_STACK is not empty do:
    current_state = Pop(OPEN_STACK)  // LIFO behavior (Depth-First)

    If current_state is a complete schedule then:
        L = Calculate_Makespan(current_state)
        If L < best_makespan then:
            best_makespan = L
            best_schedule = current_state
        Continue to the next iteration

    // Successor generation and pruning application
    Successors = Generate_Valid_Operations(current_state)
    For each operation in Successors do:
        end_time = Calculate_Completion_Time(current_state, operation)

        // Global Pruning Rule
        If end_time >= best_makespan then:
            Continue // Discard this branch (Pruning)

        new_state = Apply_Operation(current_state, operation)
        Push(OPEN_STACK, new_state)

Return best_schedule, best_makespan
```

By integrating this pruning mechanism, the DFS algorithm significantly reduces its computational overhead, making it a highly competitive exact baseline to evaluate against the strict 60 second time budget imposed in our experimental protocol.

3.2 Heuristic Search: A* Algorithm

While uninformed search methods are memory-efficient, they lack directional guidance and explore paths blindly. To optimize the exploration of the JSSP state space, an A* (A-Star) search algorithm was implemented. It is widely recognized in Artificial Intelligence for its optimal efficiency, drastically reducing the search space by prioritizing nodes that head directly toward the goal (Hadjoudj, 2026; Russell & Norvig, 2020).

A* achieves this focused search by assigning a score to every discovered state. The core equation of A* is defined as $f(n) = g(n) + h(n)$ (Russell & Norvig, 2020):

- $g(n)$ (Exact cost): The known, accumulated cost from the initial state to the current node n. In our implementation, this is the current partial makespan.

- $h(n)$ (Heuristic estimate): An educated guess estimating the remaining cost from node n to the final goal.
- $f(n)$ (Total cost): The sum of $g(n)$ and $h(n)$.

During execution, the algorithm maintains an OPEN set (a priority queue using heapq in Python). At each iteration, the state with the lowest f-cost is popped, its valid successors are generated, and they are pushed back into the OPEN set for future evaluation (Hadjoudj, 2026). The success of A* relies entirely on the heuristic function $h(n)$. To guarantee that the algorithm finds the optimal makespan, the heuristic must be *admissible*, meaning it must never overestimate the true remaining distance to the goal (Russell & Norvig, 2020).

To conduct an ablation study, two admissible heuristics were designed directly in the heuristic function, based on established lower bounds for scheduling problems (Pinedo, 2008):

- *H1* (*Maximum Remaining Job Time*): Calculates the time required to finish all remaining operations for each individual job and returns the maximum value. It is strictly admissible because the global makespan can mathematically never be less than the time required to complete the longest remaining job. *H1* is computationally fast ($O(n)$) but less informative.
- *H2* (*Maximum Machine Workload*): Calculates the total processing time pending for each machine to identify bottlenecks. *H2* returns the maximum value between *H1* and the heaviest machine workload. It remains admissible because the schedule cannot finish before the most heavily loaded machine completes its queue (Pinedo, 2008). By accounting for bottlenecks, *H2* is significantly more informative than *H1*, helping A* evaluate fewer nodes at a slightly higher computational cost per node.

The following pseudocode formalizes the implemented logic, demonstrating the integration of the evaluation function and the priority queue management:

```
Algorithm 2: A* Search for JSSP
Input: Initial state S_0, Heuristic function H (H1 or H2)
Output: Optimal schedule and its makespan

Initialize OPEN = PriorityQueue()
g(S_0) = 0, h(S_0) = H(S_0)
f(S_0) = g(S_0) + h(S_0)
Push(OPEN, <f(S_0), g(S_0), S_0>)

While OPEN is not empty do:
    current_state = Pop_Lowest_F(OPEN)

    If current_state is a goal state (all jobs completed) then:
        Return current_state.schedule, current_state.g

    Successors = Generate_Valid_Operations(current_state)
    For each operation in Successors do:
        new_state = Apply_Operation(current_state, operation)

        // Calculate costs
        new_state.g = Calculate_Max_Completion_Time(new_state)
        new_state.h = H(new_state)
        new_state.f = new_state.g + new_state.h

        Push(OPEN, <new_state.f, new_state.g, new_state>)
```

```
22:
23: Return NULL // If time limit reached or no solution exists
```

By using these heuristic lower bounds, the A* algorithm intelligently prioritizes the most promising schedules. This directional guidance, combined with the contrast between the fast evaluation of *H1* and the bottleneck awareness of *H2*, provides a robust framework to evaluate how informed search methods maximize their efficiency under the strict 60 second time constraint of our experiments.

3.3 Local Search: Multi-Start Hill Climbing Algorithm

The JSSP is an NP-hard combinatorial optimization problem characterized by a massive search space and a topology filled with multiple local optima (Talbi, 2009). While constructive methods build solutions step-by-step, local search algorithms are advantageous for the JSSP because they operate by optimizing complete, feasible operation sequences from start to finish. However, a standard Hill Climbing algorithm suffers from a critical vulnerability in this context: it converges rapidly toward the first local optimum it encounters and stagnates.

To overcome this, a Multi-Start Hill Climbing variant combined with Variable Neighborhood Descent (VND) was implemented. This specific approach is highly effective for the JSSP for two primary reasons (Talbi, 2009):

- The multi-start mechanism forces the algorithm to jump to entirely new, randomly generated initial sequences once local improvements are exhausted, ensuring continuous global exploration of the state space under a strict time budget.
- The VND technique allows the algorithm to alternate between minor perturbations for rapid descent and more aggressive perturbations to escape shallow local optima without requiring a complete restart.

In the JSSP, moving to a "neighboring" state requires modifying the execution order of operations without violating job precedence constraints. In this implementation, the neighborhood is explored by mutating the sequence through two distinct structures evaluated sequentially (Chiang & Fu, 2008):

- *Neighborhood 1* (Adjacent Swap, *N1*): Generates a small, rapidly evaluated neighborhood by swapping only adjacent elements (i and $i + 1$) in the sequence, provided they do not belong to the same job (Chiang & Fu, 2008).
- *Neighborhood 2* (Global Swap, *N2*): Acts as an escape mechanism when *N1* fails to yield improvements. It swaps any two non-adjacent operations (i and j), creating a significantly larger neighborhood that deeply alters the schedule's structure (Chiang & Fu, 2008).

During the search cycle, if a mutation in any neighborhood produces a strictly lower makespan, it is immediately accepted as the new base state, and the descent process restarts from that point. The integration of these sequential neighborhood structures alongside the global restart mechanism is formalized in Algorithm 3:

```
Algorithm 3: Multi-Start Hill Climbing with Variable Neighborhood Descent (VND)
Input: Termination criteria (e.g., maximum time or maximum restarts)
Output: best_makespan, best_schedule

1:  best_makespan = ∞
2:  best_schedule = NULL
3:
4:  While termination criteria not met do:
5:      // 1. Random Start (Restart)
```

```
    current_sequence = generate_random_sequence()
    current_makespan = decode_schedule(current_sequence)
    improved = TRUE

    // 2. Greedy Hill-Climbing Loop (VND)
    While improved is TRUE and termination criteria not met do:
        improved = FALSE
        best_neighbor_makespan = current_makespan
        best_neighbor_sequence = NULL

        // Evaluate Neighborhood 1 (Fast)
        neighbors = get_neighbors_n1(current_sequence)
        For each nb in neighbors do:
            If decode_schedule(nb) < best_neighbor_makespan then:
                best_neighbor_makespan = decode_schedule(nb)
                best_neighbor_sequence = nb

        // If N1 fails, switch to Neighborhood 2 (Deep)
        If best_neighbor_sequence is NULL then:
            neighbors_n2 = get_neighbors_n2(current_sequence)
            For each nb in neighbors_n2 do:
                If decode_schedule(nb) < best_neighbor_makespan then:
                    best_neighbor_makespan = decode_schedule(nb)
                    best_neighbor_sequence = nb

        // Apply the best found move
        If best_neighbor_sequence is not NULL then:
            current_sequence = best_neighbor_sequence
            current_makespan = best_neighbor_makespan
            improved = TRUE

    // 3. Update Global Best Solution
    If current_makespan < best_makespan then:
        best_makespan = current_makespan
        best_schedule = current_sequence

Return best_makespan, best_schedule
```

Combining the multi-start mechanism with VND transforms a traditional local search into a highly dynamic optimization engine. It strikes a precise balance between broad global exploration and rapid local exploitation, ensuring the algorithm never stagnates in sub-optimal valleys. This relentless cycle of random restarts and greedy descents is explicitly designed to extract the maximum possible value from the strict 60 second computational budget, setting a robust foundation for the performance and convergence metrics evaluated in Section 4.

4. Experimental Setup and Results Analysis

4.1 3 Jobs x 3 Machines JSSP base instance

To evaluate the performance of the proposed search methodologies, each algorithm was executed on the defined 3×3 Job-Shop Scheduling Problem (JSSP) instance under a standardized testing environment. All algorithms were subjected to a strict computational time budget of 60.0 seconds. Performance was quantitatively assessed

across key operational criteria: solution quality (*makespan*), total execution time, time required to locate the optimal solution, node expansion count (search effort), peak memory/frontier footprint, and success rate. The quantitative results obtained across all four experimental evaluations are summarized in Table 1:

Table 1. Performance Comparison of Search Algorithms on the 3 × 3 JSSP Instance

Algorithm	**Depth-First Search (DFS)**	**A* Search**		**Multi-Start Hill Climbing**
Heuristic / Mode	Upper Bound Pruning	H1 (Max Job Time)	H2 (Machine Workload)	Variable Neighborhood Descent
Best Makespan	12	12	12	12
Execution Time (s)	0.0085	0.0057	0.0025	60.0002
Time to Best (s)	0.0018	0.0057	0.0025	0.0006
Success Rate (%)	100%	100%	100%	100%
Expanded Nodes (Iterations)	1,263	272	147	6,192,874
Max Frontier / Memory	10	448	250	140,189 (Restarts)

All evaluated algorithms achieved a 100.0% success rate and successfully converged to the global optimal makespan of 12 time units. This uniform convergence confirms the correct formulation of each approach and validates that heuristics $H1$ and $H2$ remained strictly admissible throughout the search space.

The ablation study between $H1$ and $H2$ within the A^* framework highlights the critical impact of heuristic informativeness on computational efficiency. While $H1$ (Maximum Remaining Job Time) provided a valid lower bound, its inability to account for machine contention required expanding 272 nodes. Conversely, $H2$ incorporated bottleneck machine workloads into its estimation, achieving an 85% reduction in node expansions (147 nodes) and cutting peak memory usage nearly in half (250 versus 448 states). Consequently, $H2$ reduced total execution time to 0.0025 seconds, proving that tighter lower bounds dramatically compress the search frontier.

A broader comparison reveals distinct operational trade-offs across search paradigms. Uninformed DFS with pruning demonstrated remarkable memory efficiency, requiring a maximum frontier of only 10 states; however, its lack of directional guidance led to $1{,}263$ node expansions. In contrast, Multi-Start Hill Climbing leveraged its stochastic local search operators to discover the optimal makespan in just 0.0006 seconds, the fastest among all methods. Unbound by priority queue management, it executed over $140{,}000$ global restarts and evaluated 6.19 million state transitions within the allocated time budget, underscoring the raw throughput of local search mechanisms.

To determine the superior approach for this 3×3 JSSP instance, computational speed must be weighed against mathematical exactness. Although Multi-Start Hill Climbing identified the optimal solution earliest, its stochastic nature lacks an internal stopping criterion to formally prove optimality, forcing it to run until the time limit expired. Therefore, A^* Search with $H2$ emerges as the most robust methodology for small scale JSSP instances, systematically proving global optimality with minimal node expansions and minimal computational overhead.

Figure 2 illustrates the Gantt diagram corresponding to the schedule generated by A^* with $H2$. As shown, the operations are tightly packed across all three machines, eliminating unnecessary idle time and successfully completing all jobs within the certified minimum makespan of 12 time units.

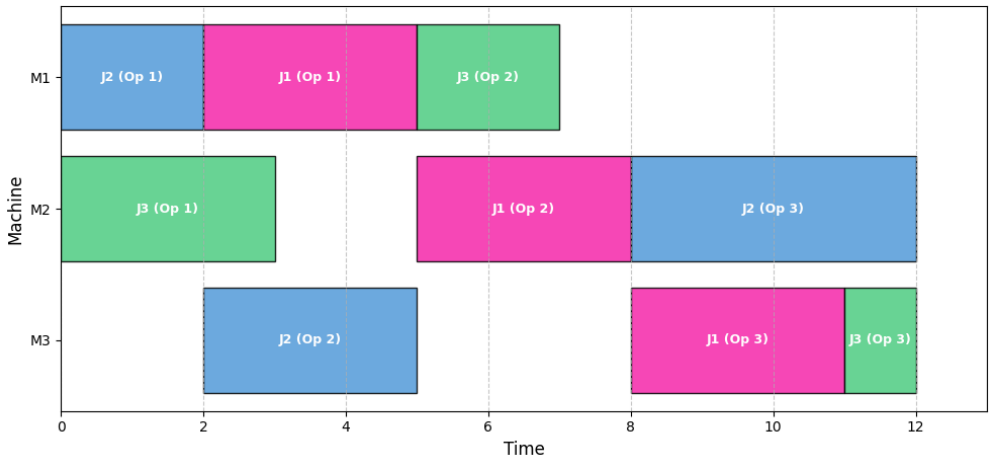


Figure 2. Optimal JSSP Schedule Generated by A* (*H2*) with a Makespan of 12 Time Units

4.2 Twenty Instances and Two Benchmarks

To evaluate how well the algorithms scale beyond the small base case, the testing environment was expanded to 22 total instances. A random generator with a fixed seed (*random.seed(42)*) was used to ensure exact reproducibility. This created 20 synthetic instances categorized by size: 10 Easy (5×5), 8 Medium (10×10), and 2 Hard (15×15). In addition, algorithm performance was tested against two standard benchmark problems taken from the OR-Library (Beasley, 1990):

- *ft10* (Fisher and Thompson 10x10): A classic, highly constrained problem also known as mt10.
- *la36* (Lawrence 15x15): A larger problem that heavily challenges memory usage.

All tests ran under a strict 60 second time limit, measuring *Best Makespan*, *Time to Best*, *Success Rate*, *Expanded Nodes*, and *Max Frontier Size*.

The DFS implementation showed the classic trade-off between low memory usage and blind search behavior. The simple global pruning rule (*if end_time >= best_makespan: continue*) kept memory usage very low: the *Max Frontier Size* stayed near 51 states in *Easy* instances and never exceeded 1,576 states in *Hard* instances. Because DFS does not calculate complex heuristics or reorder data structures, it processed a massive number of nodes, over 15 million expansions in *Easy* instances within 60 seconds. However, searching blindly caused severe problems as sizes grew. On *Medium*, *Hard*, and benchmark instances (*ft10* and *la36*), DFS timed out (*Timed Out: TRUE*). It returned a makespan of 2,847 for *ft10* and 9,146 for *la36*, showing that for dense problems ($N \times M \geq 100$), DFS only serves as a weak initial baseline generator.

The ablation study for informed search compared two admissible heuristics (H1: *Max Remaining Job Time* vs. H2: *Max Machine Workload*) across three weight factors ($W = 1.0,1.5,2.0$). The results confirm that H2 is better. By looking at machine bottlenecks instead of just job times, H2 provides a much tighter lower bound $h(n)$. This allowed standard A* ($W = 1.0$) to prune useless branches earlier on *Easy* instances, expanding far fewer nodes than H1 to prove optimality. Adding weights ($W = 1.5,2.0$) turned Weighted A* into a greedy search. By boosting the heuristic value, weighted versions found good solutions in fractions of a second (e.g., *Time to Best* of 0.0107s on *Easy_5* with H2 and $W = 2.0$). This sacrificed guaranteed optimality but greatly improved search speed. However, the main bottleneck for A* was managing its priority queue (*open_set*). On *Medium*, *Hard*, and benchmark problems, the large branching factor forced the queue (*Max Frontier Size*) to store 1 to 3 million states.

The algorithm spent most of its 60-second budget copying dictionaries (*.copy()*) and reordering the heap, hitting the time limit without improving its initial solutions on ft10 or la36.

Multi-Start Hill Climbing (MSHC) proved to be the most resilient and scalable method against the large search space of the JSSP. Unlike tree search methods (DFS, A*), MSHC saves each state as a full sequence of operations and calculates its final schedule dynamically using a decoder (*decode_schedule*). The Variable Neighborhood Descent (VND) approach worked very well: the algorithm first tries quick adjacent swaps (*Neighborhood 1*). If it gets stuck in a local minimum, it switches to global non-adjacent swaps (*Neighborhood 2*). The benefit of this two-step design was clear on the benchmarks:

- On *ft10*, MSHC evaluated 565,201 full neighbor sequences in 60 seconds, finding a makespan of 1,050 (close to the best-known literature optimal of 930).
- On *la36*, despite the much larger problem size, the algorithm evaluated over 261,000 structures, reporting a makespan of 1,585.

For memory usage, *Max Frontier Size* was replaced by the number of global *Restarts* (ranging between 1 and 21 on heavy instances). With a constant $O(1)$ memory requirement per neighbor, MSHC never suffered from the memory slowdown that stopped A*. To maintain concise executive reporting and avoid a cluttered 66 row table, the individual runs of the 20 synthetic instances were consolidated into average metrics grouped by complexity scale (Easy, Medium, Hard), while benchmark instances are presented individually. A comprehensive overview of all consolidated empirical performance metrics across instance scales and standard benchmarks is provided in Appendix A (Table A1).

A visual comparison of the algorithm scalability and Best Makespan performance across all categories and benchmarks is illustrated in Figure A2 (Appendix A). While A* and Weighted A* match the optimal makespan in small instances, their performance drops due to time outs on larger sizes. In contrast, MSHC systematically maintains the lowest makespan values across Medium, Hard, ft10, and la36 instances.

5. Conclusions

This project evaluated four search algorithms: Depth-First Search (DFS), A* Search with the *H1* and *H2* heuristics, Weighted A*, and Multi-Start Hill Climbing (MSHC) to solve the Job-Shop Scheduling Problem (JSSP) across 22 instances under a strict 60 second time limit. For small scale problems (3x3 to 5x5 instances), exact informed search methods proved to be the most effective approach. Specifically, A* equipped with the *H2* heuristic based on machine workload successfully guaranteed global optimality with the minimum number of expanded nodes and lower computational overhead, while Weighted A* achieved the fastest solution discovery.

However, as problem complexity increased to larger benchmarks such as *ft10* and *la36*, the methods encountered severe scalability limits. Standard A* timed out due to the high computational cost of managing its priority queue across millions of generated states, while DFS expanded nodes blindly without finding high-quality schedules. In contrast, Multi-Start Hill Climbing (MSHC) combined with Variable Neighborhood Descent (VND) proved to be the most resilient and scalable methodology. MSHC systematically achieved the best makespan values on all medium, hard, and standard benchmark instances.

Ultimately, these findings highlight a key trade-off between mathematical exactness and practical scalability in combinatorial optimization. While exact algorithms like A* with *H2* are ideal for small problems where optimal solutions must be proven, local search metaheuristics like MSHC are vastly superior for tackling large, real world scheduling instances within tight execution constraints.

6. AI-Use Statement

1. Tools Used:

- ChatGPT (GPT-4o) / Gemini 1.5 Pro / Gemini 1.5 Flash.

2. What Generative AI Was Used For:

- Clarifying theoretical concepts regarding JSSP modeling, heuristic admissibility, and local search topology.
- Assisting in debugging Python search algorithms and optimizing data structure efficiency.
- Improving writing style, academic tone, and grammatical clarity in the written report.
- Neighborhood formulations ($N1$ Adjacent Swap and $N2$ Global Swap).
- Formatting pseudocode and references.

3. What Was Designed and Implemented Entirely by the Team:

- State, action, and cost modeling for the Job-Shop Scheduling Problem state space.
- Mathematical design and admissibility proof of heuristics $H1$ (Max Job Time) and $H2$ (Max Machine Workload).
- Experimental protocol design, including fixed seeds, 60 second execution limit, and automated logging scripts.
- Empirical validation and critical discussion of experimental trade-offs.

4. How Correctness Was Verified:

- Gantt diagram correctness.
- Running ablation studies ($H1$ vs. $H2$, $N1$ vs. $N2$) to confirm expected node expansion drops.
- Cross-verifying optimal makespan values against known JSSP literature bounds.
- Automating reproducibility tests with execution scripts across multiple independent runs.

5. Parts That Cannot Be Defended Without External Help:

- None. The team understands, reproduces, and hopes to be prepared to defend all modeling choices, algorithms, code implementations, and experimental results presented in this report.

7. References

Beasley, J. E. (1990). *OR-Library: Job-shop scheduling* [Data set]. Brunel University London. https://people.brunel.ac.uk/~mastjjb/jeb/orlib/files/jobshop1.txt

Chiang, T. C., & Fu, L. C. (2008). A rule-centric memetic algorithm to minimize the number of tardy jobs in the job shop. *International Journal of Production Research*, 46(24), 6913-6931.

Hadjoudj, M. I. (2026). *A* search algorithm: Pathfinding in games and AI*. Operations Research Engineering.

Mencía, C., Sierra, M. R., Salido, M. A., Escamilla, J., & Varela, R. (2015). Solving the job shop scheduling problem with operators by depth-first heuristic search enhanced with global pruning rules. *AI Communications, 28*(1), 53-69. https://doi.org/10.3233/AIC-140630

Pinedo, M. L. (2008). *Scheduling: Theory, algorithms, and systems* (3rd ed.). Springer.

Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

Talbi, E. G. (2009). *Metaheuristics: From design to implementation*. John Wiley & Sons.

Yamada, T., & Nakano, R. (1997). Job-shop scheduling. En A. M. S. Zalzala & P. J. Fleming (Eds.), *Genetic algorithms in engineering systems* (pp. 134–160). The Institution of Electrical Engineers.

APPENDIX A

Note on Table A1: Rather than presenting raw output for all 66 experimental runs (22 instances × 3 algorithms), which would create an overly redundant table, the 20 synthetic instances are presented as averaged metrics grouped by difficulty category (Easy: 10 instances, Medium: 8 instances, Hard: 2 instances). The two OR-Library standard benchmarks (*ft10* and *la36*) are reported individually. Furthermore, while MSHC found optimal or near-optimal solutions in sub-second times, its Timed Out metric reports TRUE because stochastic local search continuously executes global random restarts until the 60 second timer expires.

Table A1. Empirical Performance Comparison across Algorithms and Instance Scales (60 seconds budget)

Category / Instance	Algorithm	Best Makespan	Time to Best (s)	Expanded Nodes (Avg.)	Max Frontier Size	Timed Out
Easy (5x5, 10 instances average)	DFS	44.1	5.20	> 15,000,000	51	TRUE
	A* (W=1.0, H1/H2)	44.1	N/A	~ 800,000	~ 3,000,000	TRUE
	Weighted A* (W=2.0, H2)	**44.0**	**0.0107**	~ 3,600,000	~ 330,000	TRUE
	MSHC	44.1	0.0250	~ 2,300,000	2,900 (Restarts)	TRUE
Medium (10x10, 8 instances average)	DFS	364.5	28.50	~ 11,500,000	451	TRUE
	A* / Weighted A*	N/A	N/A	~ 250,000	~ 2,600,000	TRUE
	MSHC	**97.8**	**0.0500**	~ 600,000	19 (Restarts)	TRUE
Hard (15x15, 2 instances average)	DFS	922.0	11.10	~ 9,000,000	1,576	TRUE
	A* / Weighted A*	N/A	N/A	~ 50,000	~ 700,000	TRUE
	MSHC	**182.0**	**0.1200**	~ 284,000	1 (Restarts)	TRUE
Benchmark ft10 (10x10)	DFS	2,847	28.76	11,256,512	451	TRUE
	A* / Weighted A*	N/A	N/A	~ 130,000	~ 1,100,000	TRUE
	MSHC	**1,050**	**0.2100**	565,201	9 (Restarts)	TRUE
Benchmark la36 (15x15)	DFS	9,146	1.95	8,831,544	1,576	TRUE
	A* / Weighted A*	N/A	N/A	~ 48,000	~ 680,000	TRUE
	MSHC	**1,585**	**0.4500**	261,511	1 (Restarts)	TRUE

Note on Figure A2: Figure A2 illustrates the Best Makespan achieved by each algorithm across synthetic instance groups and standard benchmarks under a 60 second limit. Missing bars for A* / Weighted A* in medium and hard instances represent computational time outs (N/A) caused by priority queue memory constraints.

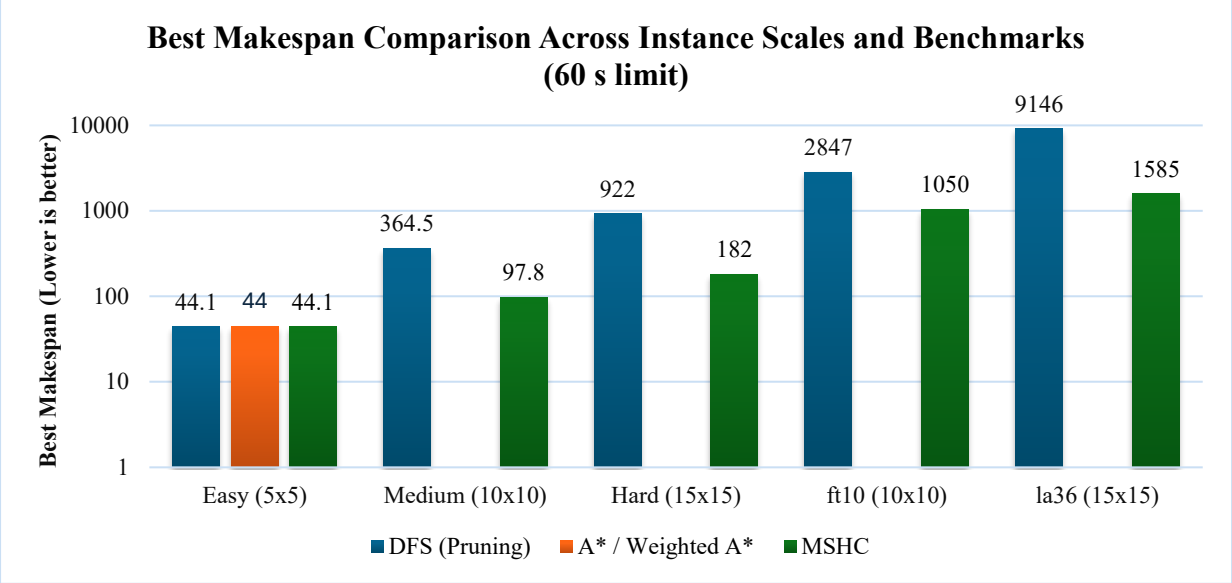


Figure A2. Best Makespan comparison across instance scales and literature benchmarks under a 60 second computational budget